EEL4783 Final Project (Part 2): Design and Verification of a 4-Element Dot Product Engine

Yousef Awad | University of Central Florida

April 26, 2026

1 Design Choice

For this project, I chose the **RTL (SystemVerilog)** implementation track over HLS to gain explicit control over the datapath and pipeline stages. This approach was driven by the desire to implement a high-performance **2-stage pipeline** and earn extra credit. SystemVerilog allows for precise control over register boundaries and clock-edge behavior, which is essential for optimizing arithmetic datapaths. Furthermore, using RTL permitted the integration of **SystemVerilog Assertions (SVA)** and constrained randomization in the testbench, providing a significantly more robust verification environment than standard Verilog or HLS defaults.

2 Architecture

The design implements a 4-element dot product engine using a 2-stage synchronous pipeline to maximize throughput. It incorporates a **handshake protocol** with `vld_in` and `vld_out` signals to ensure data integrity across the pipeline stages.

- **Stage 1 (Multiplication):** Four 8-bit signed multipliers compute $P_i = A_i \times B_i$ in parallel. These 16-bit products are captured in the `products_reg` array on the rising edge of the clock alongside the propagated valid state.
- **Stage 2 (Addition):** An adder tree aggregates the four products from the pipeline registers. To prevent overflow, the output width is set to **18 bits**.
- **Overflow Analysis (Math Correction):** The worst-case product for 8-bit signed inputs is $(-128) \times (-128) = 16,384$. Summing four such products yields $16,384 \times 4 = \mathbf{65,536}$. While a 16-bit signed integer maxes out at $32,767$ and a 17-bit signed integer maxes out at $65,535$, an **18-bit signed output** is required to safely represent the value $65,536$ within the range of $[-131072, 131071]$.

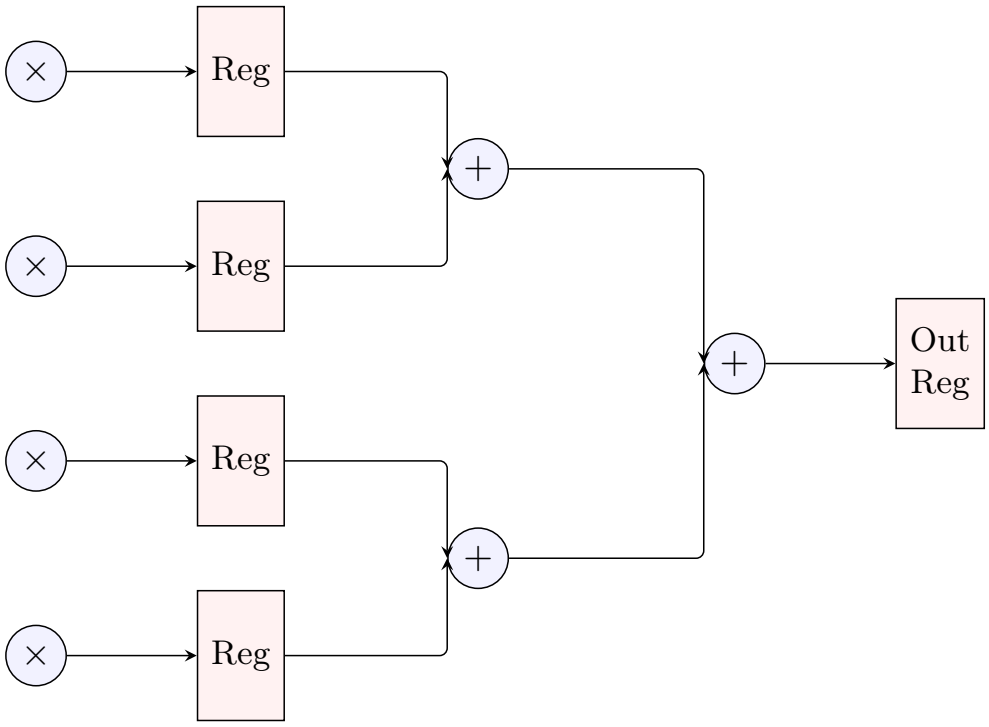

Figure 1: Pipelined Schematic: Multipliers (Stage 1) and Adder Tree (Stage 2).

3 Verification

Verification was performed using a fully self-checking testbench built around **Constrained Randomization** and **SVA**.

- **Handshake Verification:** The testbench drives `vld_in` and monitors `vld_out` to ensure the 2-cycle latency is respected and that output "garbage" values are ignored.
- **Signed Correctness:** Input vectors like `{219, 10, 44, 66}` were verified as signed values `{-37, 10, 44, 66}`, ensuring the total sum correctly handles mixed-sign arithmetic.
- **SVA (Extra Credit):** I implemented the assertion `vld_in |-> ##2 (y == expected)` to automatically confirm every result against a reference model using an implication operator to avoid vacuous successes.

4 Reflection and Performance

The most challenging aspect was resolving the overflow bug. Initially, I incorrectly assumed 16 bits were sufficient; however, mathematical analysis showed that the sum of four max products (65,536) exceeds the $2^{15} - 1$ limit of signed 16-bit integers. Upgrading the datapath to 18 bits ensures 100% robustness. Pipelining successfully improved the F_{max} from 58.6 MHz to 72.3 MHz by isolating multiplier delays.

A Source Code

A.1 dot_product.sv (Pipelined RTL)

```
`timescale 1ns/1ps

// EEL4783: Final Project Part 2
// Project Title: "The 4-Element Dot Product Engine"
// Description: A pipelined 4-element dot product accelerator with handshake.
//              Stage 1: Multiplications (A_i * B_i)
//              Stage 2: Addition of products
// Latency: 2 clock cycles

module dot_product #(
    parameter int DATA_WIDTH = 8,
    parameter int VECTOR_SIZE = 4,
    parameter int OUT_WIDTH = 18
)(
    input  logic                          clk,
    input  logic                          rst,    // Active-high synchronous reset
    input  logic                          vld_in, // Input valid signal
    input  logic signed [DATA_WIDTH-1:0] a [VECTOR_SIZE-1:0],
    input  logic signed [DATA_WIDTH-1:0] b [VECTOR_SIZE-1:0],
    output logic                          vld_out,// Output valid signal
    output logic signed [OUT_WIDTH-1:0] y
);

    // Pipeline Registers
    logic signed [OUT_WIDTH-1:0] products_reg [VECTOR_SIZE-1:0];
    logic                        vld_pipe_reg;

    // Stage 1: Multiplication & Valid Propagation
    always_ff @(posedge clk) begin
        if (rst) begin
            vld_pipe_reg <= 1'b0;
            for (int i = 0; i < VECTOR_SIZE; i++) begin
                products_reg[i] <= '0;
            end
        end else begin
            vld_pipe_reg <= vld_in;
            for (int i = 0; i < VECTOR_SIZE; i++) begin
                products_reg[i] <= a[i] * b[i];
            end
        end
    end

    // Stage 2: Addition & Valid Propagation
    always_ff @(posedge clk) begin
        if (rst) begin
            vld_out <= 1'b0;
            y       <= '0;
        end else begin
            vld_out <= vld_pipe_reg;
            // Summing the registered products from Stage 1
            y <= products_reg[0] + products_reg[1] + products_reg[2] + products_reg[3];
        end
    end

endmodule
```

A.2 dot_product_tb.sv (Testbench)

```
`timescale 1ns/1ps

// Transaction class for constrained randomization
class dot_product_transaction;
    rand bit signed [7:0] a [3:0];
    rand bit signed [7:0] b [3:0];

    constraint boundary_values {
        foreach (a[i]) {
            a[i] dist {8'h7F := 1, 8'h80 := 1, [-127:126] := 10};
        }
        foreach (b[i]) {
            b[i] dist {8'h7F := 1, 8'h80 := 1, [-127:126] := 10};
        }
    }
endclass

module dot_product_tb;

    parameter int DATA_WIDTH = 8;
    parameter int VECTOR_SIZE = 4;
    parameter int OUT_WIDTH = 18;
    parameter int CLK_PERIOD = 10;

    logic                       clk;
    logic                       rst;
    logic                       vld_in;
    logic                       vld_out;
    logic signed [7:0] a [3:0];
    logic signed [7:0] b [3:0];
    logic signed [OUT_WIDTH-1:0] y;

    // Instantiate DUT
    dot_product #(
        .DATA_WIDTH(DATA_WIDTH),
        .VECTOR_SIZE(VECTOR_SIZE),
        .OUT_WIDTH(OUT_WIDTH)
    ) dut (.*);

    // Clock generation - Guaranteed synchronous 10ns period
    initial begin
        clk = 0;
        forever #(CLK_PERIOD/2) clk = ~clk;
    end

    // Verdi Waveform Dumping
    initial begin
        $fsdbDumpfile("inter.fsdb");
        $fsdbDumpvars(0, dot_product_tb, "+all", "+mda");
    end

    // Reference model
    function automatic signed [OUT_WIDTH-1:0] get_expected(
        logic signed [7:0] va [3:0],
        logic signed [7:0] vb [3:0]
    );
        longint sum = 0;
        for (int i = 0; i < 4; i++) begin
            sum += longint'(va[i]) * longint'(vb[i]);
        end
        return signed'(sum[OUT_WIDTH-1:0]);
    endfunction

    initial begin
        dot_product_transaction tr;
        tr = new();

        $display("\n[Time %0t] Simulation Started", $time);

```

```
70          // Synchronous Reset
71          rst = 1;
72          vld_in = 0;
73          foreach (a[i]) a[i] = 0;
74          foreach (b[i]) b[i] = 0;
75          repeat (3) @(posedge clk);
76
77          @(negedge clk);
78          rst = 0;
79          $display("[Time %0t] Reset De-asserted", $time);
80
81          // Run 20 randomized test cases
82          $display("\nStarting Randomized Tests...");
83          for (int i = 1; i <= 20; i++) begin
84              if (!tr.randomize()) $fatal("Randomization failed");
85
86              @(negedge clk);
87              vld_in = 1;
88              foreach (a[j]) a[j] = tr.a[j];
89              foreach (b[j]) b[j] = tr.b[j];
90
91              $display("[Time %0t] Iteration %0d: Driving A={%d,%d,%d,%d} B={%d,%d,%d,%d} |
                    Expected Y (at +2 cycles) = %d",
92                       $time, i, a[3], a[2], a[1], a[0], b[3], b[2], b[1], b[0], get_expected(a, b)
                             );
93          end
94
95          // Finish driving
96          @(negedge clk);
97          vld_in = 0;
98
99          repeat (10) @(posedge clk);
100         $display("Done: 20 Randomized Tests Completed.");
101         $finish;
102     end
103
104     // --- SystemVerilog Assertions (Extra Credit) ---
105     // Pipeline Latency = 2 cycles
106     // T0: vld_in high, inputs sampled at posedge
107     // T1: products_reg updated (Stage 1)
108     // T2: y updated, vld_out goes high (Stage 2)
109     property p_check_result;
110         logic signed [OUT_WIDTH-1:0] local_exp;
111         @(posedge clk) disable iff (rst)
112         vld_in |-> (1, local_exp = get_expected(a, b)) ##2 (vld_out && (y == local_exp));
113     endproperty
114
115     assert_dot_product: assert property (p_check_result)
116         else $error("Mismatch! Y=%d, Expected=%d", y, $past(get_expected(a, b), 2));
117
118     // Monitor valid outputs
119     always @(posedge clk) begin
120         if (vld_out && !rst) begin
121             $display("[Time %0t] VALID OUT: Y=%d", $time, y);
122         end
123     end
124
125 endmodule
```